

An LLM-Assisted Method for Completeness Review of Software Requirements Specifications Using Related Artifacts

Haruki Hashimoto

*Graduate School of
Information Science and Technology
The University of Osaka
Osaka, Japan
har-hsmt@ist.osaka-u.ac.jp*

Shinji Kusumoto

*Graduate School of
Information Science and Technology
The University of Osaka
Osaka, Japan
kusumoto@ist.osaka-u.ac.jp*

Shinsuke Matsumoto

*Graduate School of
Information Science and Technology
The University of Osaka
Osaka, Japan
shinsuke@ist.osaka-u.ac.jp*

Yoshihiro Saito

*O.B.System Inc.
Osaka, Japan
y-saito@obs.co.jp*

Yoshiteru Kita

*O.B.System Inc.
Osaka, Japan
kita@obs.co.jp*

Kensuke Takeda

*O.B.System Inc.
Osaka, Japan
kensuke-takeda@obs.co.jp*

Abstract—A software requirements specification (SRS), as the final deliverable of the early phases of software development, strongly affects the subsequent design, implementation, and testing phases, and ensuring its quality is a critical issue. In particular, a completeness review, which checks whether all the necessary requirements are described in the SRS without omission, is essential. In practice, however, related artifacts such as meeting minutes are diverse, and referring to them exhaustively and manually is difficult. This paper proposes a framework for the completeness review of SRSs, focusing on the consistency between an SRS and its related artifacts. Specifically, the framework constructs an integrated reference model that consolidates the requirements-related information in the related artifacts and reviews the completeness of the SRS by comparing it with this model. We implemented the proposed framework using a large language model (LLM) and evaluated it using an SRS and related artifacts from a real software development project. The results showed that the framework enables a completeness review based on the consistency with related artifacts. In the LLM-based implementation, candidate missing requirements and candidate unsubstantiated requirements were detected stably, with a recall of 100% and a precision of 90% or above, confirming the usefulness of an LLM as a means of implementing the framework. The analysis of the detected candidates also confirmed that, in applying the framework, it is important that requirements-related decisions be adequately recorded in the related artifacts.

Index Terms—Software Requirements Specifications, Completeness Review, Consistency with Related Artifacts, Large Language Models, Empirical Evaluation

I. INTRODUCTION

A software requirements specification (SRS) is a document that specifies a software product (or a set of software products) that performs certain functions in a specific operating environment [1]. An SRS is developed through a series of activities including eliciting, analyzing, organizing, and documenting requirements, together with multiple meetings and consensus

building among stakeholders. This process includes a requirements review, which aims at the early detection of defects in the SRS, such as missing descriptions and ambiguity. Because the quality of an SRS affects the quality of the software to be developed and even the success of the project itself [2], [3], the requirements review is an essential activity in software development.

In a requirements review, the content of the SRS is examined from various perspectives, such as the quality characteristics that ISO/IEC/IEEE 29148 [1] requires of the set of requirements in an SRS. Among these characteristics is completeness, which indicates whether the SRS describes all the necessary information without omission; the details of the characteristics are described in Section II-B.

Meanwhile, requirements reviews in real-world development projects face many challenges. One of them is that conducting a requirements review requires considerable time. Requirements reviews are performed under time and budget constraints, and it has been pointed out that this time cost may degrade the quality of the reviews [4]. Requirements reviews also often require domain knowledge of the target business operations and the applicable regulations, and review quality therefore tends to depend on the knowledge and experience of individual reviewers. As a result, variation in review outcomes and dependence on particular individuals have also been raised as practical concerns. Porter et al. likewise noted that commonly used inspection methods force reviewers to rely on non-systematic techniques when searching for defects [5]. One factor behind these challenges is the difficulty of manually and exhaustively checking the correspondence between the content of the SRS and the artifacts on which it was based (hereafter, related artifacts) [6].

To address these challenges, various studies have aimed to support requirements reviews. In recent years, studies applying

large language models (LLMs), which perform well on natural language processing tasks, have also emerged. For example, Krishna et al. [7] used LLMs to evaluate the internal quality of SRSs, such as ambiguity and consistency, treating each SRS as an independent artifact. However, research on completeness reviews that focus on the consistency between an SRS and its related artifacts remains limited [8].

In this paper, we propose a framework for reviewing the completeness of the requirements in an SRS on the basis of its consistency with related artifacts. Rather than evaluating the SRS as an independent artifact, the proposed framework captures completeness from the perspective of whether the content agreed upon and discussed in the related artifacts, such as meeting minutes, is reflected in the SRS without omission. Specifically, the framework constructs a reference model that integrates the requirements-related information contained in the related artifacts (hereafter, the integrated reference model (IRM)) and establishes correspondences between this model and the SRS, thereby identifying missing requirements and requirements whose supporting evidence is unclear. A key characteristic of the framework is that it supports the comparison and checking involved in a completeness review rather than generating an SRS. In this study, we also propose the use of LLMs as a practical means of implementing the framework.

Furthermore, we implemented the proposed framework using an LLM and conducted an experiment in which it was applied to documents (an SRS and related artifacts) created in a real software development project. As a result, high recall and precision were obtained stably in detecting candidate missing requirements and candidate unsubstantiated requirements, confirming the usefulness of an LLM as a means of implementing the framework. On the other hand, the detected candidates had not been recognized as problems in the real project and stemmed from decisions that were not recorded in the related artifacts. This result highlights the importance of adequately recording requirements-related decisions in the related artifacts when applying the framework.

II. BACKGROUND AND RELATED WORK

A. Requirements Definition

Requirements definition is one of the early phases of the software development process and is the activity of clarifying stakeholder needs as software requirements. ISO/IEC/IEEE 12207 [9], the international standard that provides the framework for the overall software development process, states that the purpose of requirements definition is to define, as requirements, the software functions and constraints that satisfy the needs of stakeholders. Requirements definition consists of multiple tasks: defining requirements, analyzing requirements, and managing requirements.

First, the requirements are defined. Specifically, the necessary functions are formulated as requirements, and their attributes and constraints are defined. Next, the requirements are analyzed. In this task, the content of the defined requirements is analyzed, and their validity is verified from various perspectives. Finally, the requirements are managed.

This task involves building consensus among stakeholders on the defined requirements and maintaining traceability between the requirements and other artifacts. Requirements definition is carried out through this process.

Various challenges in requirements definition have been pointed out. Méndez Fernández et al. reported problems that commonly arise in the early phases of development, including challenges in requirements definition [10]. We list below the problems observed in industrial practice, using the terms of that study:

- Incomplete and/or hidden requirements
- Communication flaws between project team and the customer
- Moving targets (changing goals, business processes and/or requirements)
- Underspecified requirements that are too abstract
- Time boxing / Not enough time in general

These problems involve not only the quality of the requirement descriptions themselves, such as incomplete or overly abstract requirements, but also the circumstances surrounding requirements definition, such as communication with customers, changing goals and requirements, and time constraints. These problems have been shown to remain unresolved in many real-world projects. In particular, the problem of incomplete and/or hidden requirements—that is, the situation in which requirements that are actually necessary are missing or are not documented—has been reported as the most frequent problem. In this paper, we refer to this problem as the incompleteness of requirements.

B. Requirements Review

A requirements review is one of the activities included in the requirements definition process. Specifically, it corresponds to the requirements analysis task introduced in Section II-A. A requirements review examines the validity of the content of the SRS from various perspectives in order to detect defects such as missing descriptions and ambiguity at an early stage. The systematic inspection of software documents was introduced by Fagan, who reported that the use of formal design and code inspections yielded substantial improvements in software quality and productivity [11].

In general, the quality characteristics required of requirements, as specified in ISO/IEC/IEEE 29148—the international standard for requirements engineering processes and their deliverable SRSs—serve as perspectives for conducting requirements reviews. Below, we summarize the main characteristics required of the set of requirements in an SRS among the quality characteristics described in ISO/IEC/IEEE 29148.

Completeness The set of requirements describes all the necessary information and items without omission and requires no further information

Consistency The requirements are free of conflicts and overlaps with one another, and the units and terminology are used consistently

Feasibility The set of requirements can be realized within the given constraints

Comprehensibility What is expected of the software and how it relates to the system of which it forms a part is clearly described

Validatability It can be confirmed that the satisfaction of the set of requirements leads to the achievement of the needs within the constraints

Regarding requirements reviews, Méndez Fernández et al. point out that the incompleteness of requirements, the most frequently reported problem in requirements definition, arises partly because reviews from the completeness perspective are insufficient [10]. A review from the completeness perspective checks whether the SRS describes all the necessary information without omission and therefore requires referring to the related artifacts used to create the SRS. However, it has also been pointed out that manually and exhaustively grasping the correspondences among artifacts such as requirements is not realistic in terms of cost [6]. This situation is considered to be one of the factors leading to the insufficiency of reviews from the completeness perspective.

C. Completeness of an SRS

ISO/IEC/IEEE 29148 requires a set of requirements to be complete, but does not prescribe the reference against which completeness should be assessed. The requirements engineering literature has addressed this question, and we summarise it here to make precise the notion of completeness that the framework proposed in this paper deals with.

Boehm [12] states that a specification is complete to the extent that all of its parts are present and each part is fully developed, and identifies five properties that a specification must exhibit for its completeness to be assured:

- no TBD (To Be Determined) items
- no references to functions, inputs, or outputs that are not defined in the specification
- no missing specification items (for example, missing interface specifications)
- no missing functions
- no missing products (for example, test tools)

Boehm further notes that the first two of these properties can be verified by mechanical means, whereas the remaining three generally require human intuition to verify or validate. Zowghi and Gervasi [13] name this division: the first two properties concern internal completeness, which can be assessed using only the information contained in the specification, and the remaining three concern external completeness, which ensures that all the information required for defining the problem is found within the specification. They point out that external completeness cannot be defined or measured in absolute terms, since the only truly complete specification of something would be the thing itself, and propose instead the compromise of asking whether a specification is sufficiently complete, with what counts as sufficient decided relative to the type of system being built.

Assessing external completeness therefore requires a reference outside the specification, and Ferrari et al. [14] divide

it according to which reference is used. Completeness with respect to the system to be implemented, which requires a model of that system such as a domain model, they call forward functional completeness; completeness with respect to the input documents of the requirements definition process—among them preliminary specifications, reference standards, and transcripts of meetings with customers—they call backward functional completeness. A functional requirements specification achieves the latter when all the relevant concepts expressed in the input documents, and all the relevant interactions among those concepts, are treated in the specification.

D. Large Language Models

A large language model is a type of language model built from massive amounts of text data using advanced deep learning techniques. Representative LLMs include GPT, developed by OpenAI, and Gemini, developed by Google. LLMs can perform natural language processing tasks with high accuracy. In particular, for document summarization and information extraction from documents, it has been reported that LLMs can produce high-quality outputs from only a few examples or instructions and are effective even for documents requiring expert knowledge [15]. Several studies have also suggested that LLMs contribute to substantial reductions in time costs in tasks such as selecting relevant publications from a large body of literature [16].

LLMs have also been widely applied in the field of software engineering. Their applications extend not only to downstream activities such as refactoring [17], code summarization [18], and unit test generation [19] but also to the early phases of development, and studies have addressed requirements elicitation and analysis [20] and the generation of artifacts such as SRSs [7], [21]. In particular, Krishna et al. [7] evaluated the capability of LLMs in creating and revising SRSs. They reported that the generated SRSs were comparable in quality to those written by novice software engineers but lacked detailed descriptions. Furthermore, several studies have applied LLMs to the requirements review process among these early-phase activities. Specifically, some studies have aimed to use LLMs to detect contradictions among multiple requirements in an SRS [22], [23]. Krishna et al. [7] also applied LLMs to detecting ambiguity in the requirement descriptions in an SRS. Bashir et al. [24] evaluated LLMs on the same task using requirements from two companies in the railway and industrial networking sectors, and had the generated outputs assessed by domain experts.

E. Requirements Traceability

Requirements traceability (RT) refers to the state in which a requirement can be traced from the information sources that constitute its rationale to artifacts such as design, implementation, and tests, or in the reverse direction [25]. RT is further divided into two major types: pre-requirements specification (pre-RS) traceability, which concerns a requirement before it is included in the SRS, and post-requirements specification (post-RS) traceability, which concerns a requirement after it

has been included in the SRS [25]. Because understanding RT is important in various situations in software development, a variety of studies on RT have been conducted. However, most of these studies focus on post-RS traceability; studies focusing on pre-RS traceability are scarce, and the need for new research has been pointed out [8]. In this study, we use related artifacts involved in pre-RS traceability, to support the completeness review of an SRS.

III. PROPOSED FRAMEWORK

A. Objective and Positioning

The objective of the proposed framework is to support the completeness review of SRSs, which Méndez Fernández et al. identified as insufficient and as one of the causes of the incompleteness of requirements [10]. In this section, we first present a framework for the completeness review of an SRS based on related artifacts. We then describe a concrete means of implementing the framework in practice.

As discussed in Section II-C, assessing the completeness of an SRS requires a reference outside the SRS itself. This study adopts the related artifacts from which the SRS was produced as that reference; the completeness addressed here is therefore backward functional completeness. Taking the related artifacts as the reference also allows us to ask the converse question, one that completeness in the above sense does not cover: whether each requirement described in the SRS is in turn supported by the related artifacts. The review addressed in this study asks both, and is thus limited to the question of whether the SRS contains all and only the requirements for which the related artifacts provide evidence. Within this scope, the framework can target requirement descriptions in general, including functional requirements, non-functional requirements, and constraints, as long as they can be extracted from the related artifacts. In this section, however, we present only the general form of the framework; the experiment in Section IV restricts the evaluation to functional requirements.

Accordingly, the completeness review detects two types of problematic requirements: requirements for which the related artifacts provide evidence but that are absent from the SRS (hereafter, missing requirements), and requirements that are described in the SRS even though no evidence supporting their necessity can be confirmed in the related artifacts (hereafter, unsubstantiated requirements).

Note that, as described in Section II-D, generating SRSs with LLMs has recently been proposed; at present, however, it remains difficult to generate detailed SRSs of sufficient quality [7]. For this reason, we aim to support the completeness review of SRSs rather than their generation.

B. Structure of the Framework

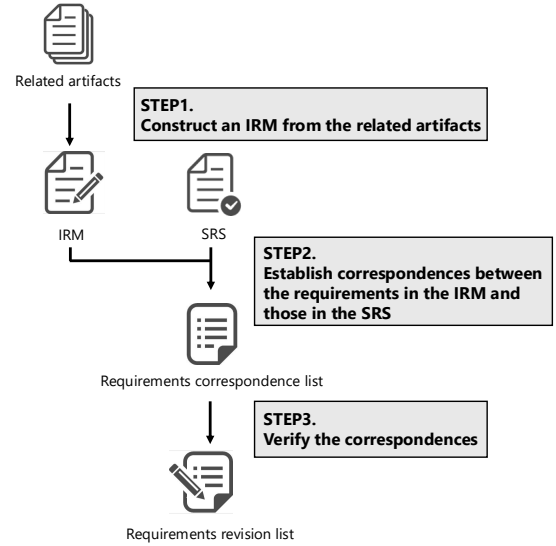


Fig. 1. Overview of the proposed framework

Fig. 1 shows an overview of the proposed framework. The framework reviews the completeness of an SRS on the basis of the information contained in related artifacts, taking the SRS and the related artifacts as inputs. It consists of the following three steps:

STEP 1. Construct an IRM from the related artifacts.

STEP 2. Establish correspondences between the requirements in the IRM and those in the SRS.

STEP 3. Verify the correspondences.

1) *STEP 1: Constructing the IRM:* Rather than comparing each related artifact directly with the SRS, the framework compares the SRS with an IRM that consolidates the information contained in the related artifacts.

The reason for introducing this consolidation step lies in the complex dependencies among related artifacts. The content discussed and examined during the requirements definition process is usually recorded across multiple related artifacts in chronological order. Consequently, each artifact is interrelated with the content of artifacts created before or after it, giving rise to complex dependencies among them. Decisions made during the requirements definition process therefore cannot be grasped by referring to a single artifact; they can be confirmed only by treating all the related artifacts as a single context and organizing the information they contain. From this viewpoint, the first step of the framework extracts the necessary requirements on the basis of the consistency of information across the related artifacts.

The concrete procedure of STEP 1 is as follows. First, each related artifact is examined, and information on the requirements under review is extracted. After information has been extracted from all the related artifacts, it is organized in chronological order. Specifically, it is checked which requirements were adopted and which were rejected, and the decision made for each requirement is traced. The requirements judged

to be required in the SRS are then listed, and the IRM containing them is obtained as the deliverable of this step. This model serves in STEP 2 as the reference against which the completeness of the SRS is checked. Note that the IRM is not itself a final SRS containing only the truly necessary requirements.

2) *STEP 2: Establishing Correspondences Between the IRM and the SRS:* To detect missing and unsubstantiated requirements, STEP 2 establishes correspondences between the requirements in the IRM constructed in STEP 1 and those described in the SRS.

To detect missing requirements, we first check, for each requirement in the IRM, whether a corresponding requirement exists in the SRS. For every requirement in the IRM, evidence supporting its necessity has been confirmed on the basis of the related artifacts. Therefore, a requirement that exists in the IRM but not in the SRS is a potential missing requirement.

Next, to detect unsubstantiated requirements, we check, for each requirement in the SRS, whether a corresponding requirement exists in the IRM. All requirements judged necessary on the basis of the related artifacts are contained in the IRM. Therefore, a requirement that exists in the SRS but not in the IRM is a potential unsubstantiated requirement.

The correspondences are thus established bidirectionally, and the results are organized into a requirements correspondence list. By referring to this list, reviewers can identify potential missing requirements (hereafter, candidate missing requirements) and potential unsubstantiated requirements (hereafter, candidate unsubstantiated requirements).

3) *STEP 3: Verifying the Correspondences:* In STEP 3, the candidate missing requirements and candidate unsubstantiated requirements identified in the requirements correspondence list are verified manually. The verification proceeds as follows.

The reviewer first re-examines the relevant parts of the related artifacts and the SRS for each candidate. If no evidence supporting the necessity of a candidate unsubstantiated requirement is found in the related artifacts, the requirement is judged to be an unsubstantiated requirement. If evidence supporting the necessity of a candidate missing requirement exists in the related artifacts and no corresponding description is found in the SRS, the requirement is judged to be a missing requirement. Through this process, a requirements revision list compiling the confirmed missing and unsubstantiated requirements is obtained as the final deliverable of the framework.

Suitable reviewers for STEP 3 are those who understand the validity of the requirements and their background, such as systems engineers responsible for requirements definition in industrial software development projects, members of the relevant business departments, and reviewers from quality assurance divisions.

C. Feasibility and Implementation Policy

The proposed framework makes it theoretically possible to conduct a completeness review by exhaustively cross-checking the related artifacts against the SRS. In practice, however, implementing the framework in its original form would be

difficult, since the cost of manually grasping the correspondences among artifacts has been pointed out to be prohibitive [6]. A practical implementation must therefore reduce the high cost associated with referring to related artifacts. In this study, we consider the application of LLMs as one such means of implementing the framework.

As described in Section II-D, LLMs perform well on tasks such as information extraction and document summarization, and their contribution to reducing time costs has also been reported [15], [16]. Building on these capabilities, we first apply an LLM to STEP 1 to reduce the cost of organizing the information contained in the related artifacts. Specifically, the LLM is given the related artifacts as input and instructed to extract and organize requirements-related information and to enumerate the requirements needed in the SRS. As a result, the IRM, in which the necessary requirements are organized, is obtained as the output of the LLM. The experiment in Section IV applies an LLM to STEP 1 in this manner.

We further consider applying an LLM to STEP 2. Because STEP 2 establishes correspondences between the requirements in the IRM and those in the SRS, its cost grows with the number of requirements handled in a project. Indeed, projects handling thousands to tens of thousands of requirements have been reported [26], [27], and in such cases the cost of STEP 2 may become impractical. To address this issue, we also apply an LLM to STEP 2. Specifically, the LLM is given the IRM and the SRS as input and instructed to establish correspondences between the requirements in the two documents. As a result, the requirements correspondence list, which records these correspondences, is obtained as the output of the LLM.

In summary, by applying LLMs to STEP 1 and STEP 2, this study reduces the implementation cost of a completeness review framework that is theoretically effective but costly to implement, thereby supporting the completeness review of an SRS.

D. Assumptions

We propose a practical implementation of a completeness review based on related artifacts, and we presuppose that the related artifacts exist in an accessible form. Related artifacts are diverse, and the difficulty of preparing and collecting them has itself been pointed out [28]. Note, however, that this study does not address how related artifacts should be prepared.

In addition, to obtain adequate review results stably through the framework, the accessible related artifacts must contain sufficient information. Consider, for example, a requirement that was decided against but whose rejection was never recorded in the related artifacts, leaving it to appear as an adopted requirement. In this situation, STEP 1 judges the requirement to be necessary for the SRS. Because the requirement does not appear in the SRS, STEP 2 then flags it as a candidate missing requirement.

The manual verification in STEP 3 can ultimately eliminate requirements that were flagged as potentially problematic but are in fact unproblematic. However, if many such requirements

arise, they may increase the review workload and induce human error. The presence of sufficient information in the accessible related artifacts is therefore an important assumption for operating the framework appropriately.

IV. EMPIRICAL EVALUATION

We conducted an experiment in which the framework proposed in Section III was applied to a dataset from a real-world software development project. As proposed there, an LLM was used to implement STEP 1 and STEP 2. Fig. 2 shows the overall flow of the experiment. The purpose of the experiment is to evaluate the practical applicability of the LLM-based implementation of the framework. This evaluation must take into account the probabilistic nature of LLM outputs. We therefore repeated the process shown in Fig. 2 five times, using the same dataset, LLM, and prompts throughout.

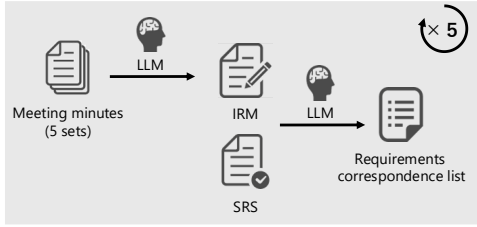


Fig. 2. Overview of the experiment

A. Dataset and LLM

As the review target, the experiment used an SRS employed in a real-world project. The target project aimed to upgrade a business support system.

In this project, five meetings were held as part of the requirements definition process. These meetings covered topics such as the project schedule and functional requirements, and minutes summarizing each meeting were produced. We therefore used these five sets of meeting minutes as the related artifacts.

The target SRS contains 29 functional requirements classified into nine functional categories. The experiment conducted the completeness review on these functional requirements.

We next describe the LLM used in the experiment. We used OpenAI’s GPT-5 via its API. This was the latest model available from OpenAI at the time the experiment was designed. Its long context window allows lengthy documents such as SRSs and meeting minutes to be handled as input. In this model, the temperature parameter, which controls the randomness of the output, is fixed at 1; we therefore used the model with this setting unchanged.

B. LLM-Based Implementation of STEP 1

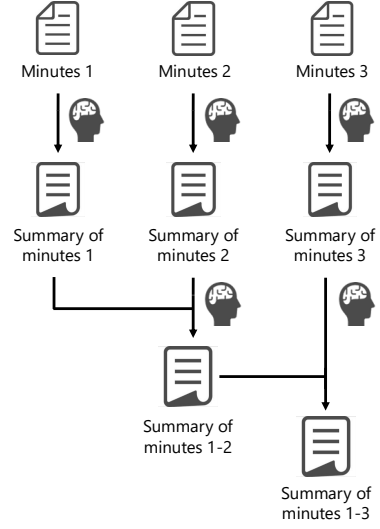


Fig. 3. Part of the STEP 1 procedure using the LLM

In STEP 1, the LLM extracts and organizes requirements-related information from the related artifacts and enumerates the requirements needed in the SRS. Because the experiment restricted the review target to functional requirements, the LLM extracts the functional requirements needed in the SRS on the basis of the related artifacts.

Fig. 3 shows part of the processing flow of this task.

First, the LLM summarizes the information on functional requirements contained in each set of meeting minutes. Specifically, the LLM is given one set of minutes together with a prompt instructing it to extract information on functional requirements from those minutes, and it outputs a summary of the minutes. This process is applied to all five sets of minutes, yielding a summary for each.

Next, the LLM integrates these summaries. Rather than merging all the summaries at once, the integration proceeds stepwise in chronological order, combining two inputs at a time. Specifically, the summaries of the first and second meeting minutes are first given to the LLM, which is instructed to extract the functional requirements needed in the SRS on the basis of these summaries. This yields a set of functional requirements reflecting the content of the first and second minutes (corresponding to “Summary of minutes 1–2” in Fig. 3).

This set of functional requirements and the summary of the third meeting minutes are then given to the LLM with the same instruction, yielding a set of functional requirements that reflects the content of the first through third minutes (corresponding to “Summary of minutes 1–3” in Fig. 3). Repeating this process until all the summaries have been integrated yields a set of functional requirements based on the content of all five sets of minutes. This set of functional requirements is used as the IRM.

C. LLM-Based Implementation of STEP 2

To implement STEP 2, the LLM judges the correspondences between the functional requirements described in the SRS and those in the IRM. To this end, the LLM is given the IRM and the SRS, together with a prompt that specifies how to assess the correspondences. The prompt instructs the LLM to check, for each functional requirement, whether a corresponding requirement exists in the other document, and to assign one of the three ratings shown in Table I.

TABLE I
THREE-LEVEL RATINGS AND THEIR MEANINGS

○	A corresponding requirement was confirmed in the other document
△	A possibly corresponding requirement was confirmed in the other document
×	No corresponding requirement was confirmed in the other document

The LLM first checks, for each functional requirement in the SRS, whether a corresponding requirement exists in the IRM. It then checks, for each functional requirement in the IRM, whether a corresponding requirement exists in the SRS. By checking in both directions, the LLM organizes the correspondences between the requirements in the two documents. The LLM outputs the result as the requirements correspondence list.

Fig. 4 shows an example of STEP 2 performed by the LLM. In Fig. 4, the parts enclosed by the red frame show the result of checking whether the requirements in the SRS have corresponding requirements in the IRM, whereas the parts enclosed by the blue frame show the result of checking whether the requirements in the IRM have corresponding requirements in the SRS.

In the example in Fig. 4, requirement E is detected as a candidate unsubstantiated requirement and requirement F as a candidate missing requirement.

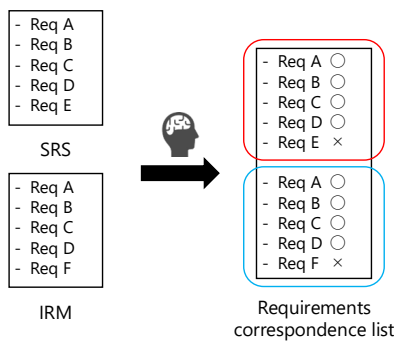


Fig. 4. Example of STEP 2

D. Overview of the Evaluation

To evaluate the practical applicability of the LLM-based implementation of the framework, we conducted the evaluation from the following three perspectives.

The first is the usefulness of an LLM as a means of implementing the framework. This evaluation was performed by verifying the IRMs and the requirements correspondence lists obtained from the LLM in the experiment, which correspond to STEP 1 and STEP 2, respectively.

The second is the validity of the detected candidates. This evaluation examined whether the candidate missing requirements and candidate unsubstantiated requirements extracted in the experiment corresponded to requirements that had actually been problematic in the real project.

The third is the time the LLM took to execute STEP 1 and STEP 2. Through this evaluation, we examined whether the LLM-based implementation of the framework can contribute to reducing the time cost of the completeness review of SRSs.

1) *Evaluation Method for the IRMs:* We evaluated the content of the IRM generated by the LLM in each trial from the perspectives of validity and stability. For validity, we checked, for each requirement in the IRM, whether the meeting minutes contain evidence justifying it as a requirement. This evaluation targeted the five IRMs obtained in the five trials and was performed by manually examining the relevant parts of the minutes. For stability, we compared the contents of the five IRMs with one another to check whether the generated results differed. Through these two evaluations, we evaluated the usefulness of the LLM in STEP 1.

2) *Evaluation Method for the Requirements Correspondence Lists:* We also evaluated the requirements correspondence lists in terms of validity and stability. The purpose of the requirements correspondence list is to present candidate missing requirements and candidate unsubstantiated requirements to the reviewer who performs the final check. To evaluate the validity of the lists, we therefore computed the recall and precision of the detection of these candidates. For this computation, the authors manually established the correspondences between the SRS and the IRM obtained in each trial, and used the result as the ground-truth correspondence data. In the experiment, requirements rated “△” or “×”—that is, those for which no clear correspondence was confirmed—were treated as candidate missing requirements or candidate unsubstantiated requirements. Accordingly, we computed recall and precision by treating “△” and “×” as Positive and “○” as Negative. For stability, we compared the recall and precision across the five requirements correspondence lists. Through these two evaluations, we evaluated the usefulness of the LLM in STEP 2.

3) *Evaluation Method for the Detected Candidates:* We examined whether the requirements detected in the experiment as candidate missing requirements and candidate unsubstantiated requirements had actually been recognized as problems in the project. Through this evaluation, we evaluated whether any issues arise when the proposed framework is applied to real data. For this evaluation, we asked engineers at the company that provided the project data to confirm the detected candidates.

4) *Evaluation Method for the Execution Time of the LLM-Based Implementation:* To evaluate the execution time of the

LLM-based implementation of the proposed framework, we conducted ten additional trials separate from the five trials used in the evaluations above. In these ten trials, the completeness review was performed with exactly the same procedure as in the previous trials, and the time the LLM took to execute STEP 1 and STEP 2 was measured. The measured times were then used to examine the contribution described above.

V. EXPERIMENTAL RESULTS

A. Evaluation Results for the IRMs

We first evaluated the validity of the requirements described in the five IRMs by referring to the meeting minutes. As a result, for most of the requirements in the IRMs, supporting evidence was confirmed in the minutes. On the other hand, we also found that the IRMs contained some requirements that should not have been included.

These were requirements that the minutes suggested were out of scope of the new development. However, a close examination of the corresponding parts of the IRMs revealed that these requirements were accompanied by notes stating that they were excluded from the new development. This suggests that the LLM performed the task while recognizing, to some extent, that these requirements were outside the scope of the new development.

Accordingly, clarifying how such requirements should be handled in the prompt may enable the output to be controlled more appropriately. Considering the above, these cases do not seriously impair the accuracy of the LLM in STEP 1, and we conclude that the LLM achieves a certain level of validity in this task.

Next, to evaluate the stability of the LLM in this task, we counted the number of requirements described in each of the five IRMs. The counts were 69, 62, 66, 62, and 65 for the first through fifth trials, respectively. These results show a variation of up to seven requirements across the trials. Below, we analyze which kinds of requirements caused the differences among the trials.

The requirements whose treatment varied across the trials fell into the following three types:

- Semantically decomposable requirements
- Requirements suggested in the minutes to be out of scope of the new development
- Requirements whose descriptions in the minutes are potentially misleading

A semantically decomposable requirement is one that can be regarded as a single unit of meaning but can also be interpreted as multiple related requirements when subdivided. For example, a description in the minutes such as “implement X, which performs A and B” falls into this type. When extracting requirements from such descriptions, the LLM sometimes treated them as a single requirement, “implementation of X,” and at other times treated “implementation of A” and “implementation of B” as individual requirements related to “implementation of X.” Such cases are considered one of the main causes of the variation in this task.

Furthermore, for requirements suggested in the minutes to be out of scope of the new development, their treatment in the IRMs also differed across the trials. These are the same cases as those mentioned in the validity evaluation above.

Finally, we also identified requirements whose descriptions in the minutes are potentially misleading. In the experiment, for one requirement whose treatment differed across the trials, the minutes contained an expression that could be interpreted as excluding the requirement from the scope. The passage in question actually meant that the requirement would be realized by a different method rather than the existing one, but the interpretation of this expression varied across the trials, which is considered to have caused the differences in its treatment in the IRMs.

In summary, in this experiment, variation across the trials was observed for particular types of requirements. We therefore conclude that challenges remain regarding the stability of the LLM in constructing the IRM.

B. Evaluation Results for the Requirements Correspondence Lists

Fig. 5 shows the recall and precision of the LLM in the task of detecting candidate missing requirements and candidate unsubstantiated requirements.

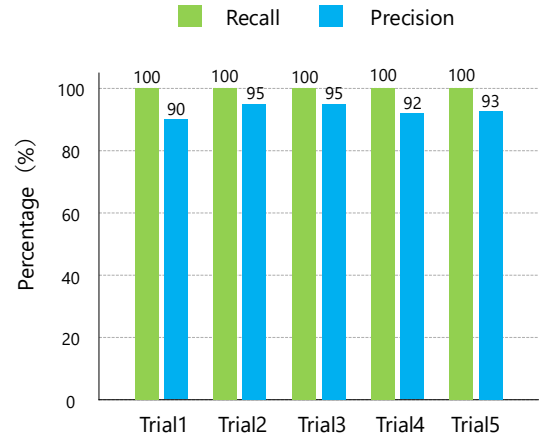


Fig. 5. Recall and precision for each trial

As shown in Fig. 5, the recall of the detection task was 100% in all five trials. This result indicates that, at least under the conditions of this experiment, the LLM detected the candidate missing requirements and candidate unsubstantiated requirements without omission.

The precision was also high, at 90% or above in all trials. This indicates that the LLM did not detect an excessive number of candidates and, on the whole, appropriately organized the correspondences between the requirements.

From these results, we conclude that the LLM exhibits high accuracy and stability in producing the requirements correspondence lists.

C. Evaluation Results for the Detected Candidates

We asked engineers at the company that provided the project data to confirm the candidate missing requirements

and candidate unsubstantiated requirements obtained in the experiment. As a result, we found that the detected candidates had not been recognized as problems in the real project. It was also confirmed that the software developed on the basis of the SRS has been operating without problems in production.

As a factor behind this, we confirmed that, in this project, decisions on requirements were made not only in the minuted meetings but also through diverse communication channels, including e-mails, phone calls, and informal meetings, and that the content of these decisions was shared and understood among the project stakeholders. This suggests that the review quality of the framework depends heavily not only on the performance of the LLM but also on the comprehensiveness of the available related artifacts.

At the same time, it is not always easy in practice to record all such informal communication comprehensively and retain it as accessible artifacts. The results of the experiment therefore suggest that, when applying an artifact-based completeness review, the scope and nature of the available artifacts must be taken into account.

D. Evaluation Results for the Execution Time of the LLM-Based Implementation

Table II shows the time the LLM took to execute STEP 1 and STEP 2 in each of the ten trials conducted separately for the execution-time measurement.

TABLE II
EXECUTION TIME OF EACH TRIAL

Trial	1	2	3	4	5
Time [s]	821	1035	1279	1336	1002
Trial	6	7	8	9	10
Time [s]	895	1284	1209	1393	824

As shown in Table II, the execution time ranged from 821 s to 1393 s, with an average of 1107.8 s, indicating that the entire process was completed in around 20 minutes in every trial. Note that we did not measure the time required to perform this process manually, and thus a direct comparison with manual work has not been made; this is a limitation of the present evaluation. Nevertheless, all the measured execution times were within a practically acceptable range, and in the target project the completeness review was performed with high repeatability using the LLM. From the above, we consider that the LLM-based implementation of the framework has the potential to contribute to reducing the time cost of the completeness review of SRSs.

VI. DISCUSSION

A. Summary and Discussion of the Evaluation Results

Through the experiment, we confirmed that an LLM has a certain level of usefulness as a means of implementing the proposed framework. In STEP 1, the LLM, on the whole, accurately organized the information in the meeting minutes and extracted the requirements needed in the SRS. In STEP 2, in the task of detecting candidate missing requirements and

candidate unsubstantiated requirements, the recall was 100% and the precision was 90% or above in all trials, with the manually created correspondences as the ground truth. From these results, we conclude that, at least under the conditions of this experiment, the LLM exhibited high accuracy and stability in STEP 2. Note, however, that these metrics were computed against the IRM constructed by the LLM from the minutes; they therefore do not indicate how well the framework detected requirements that were actually problematic in the project.

In STEP 1, variation in the number of requirements across the trials was observed, revealing a challenge in terms of stability. However, we consider that this challenge can be addressed to some extent by refining the prompts and by clarifying the descriptions in the related artifacts. Indeed, such measures are considered effective for the three types of requirements that impaired the stability of STEP 1 in the experiment. Specifically, for “semantically decomposable requirements” and “requirements whose descriptions in the minutes are potentially misleading,” describing the granularity and intent of the requirements more clearly in the related artifacts and eliminating ambiguity could address the issue. For “requirements suggested in the minutes to be out of scope of the new development,” explicitly specifying in the prompt the conditions for including requirements in the IRM could likewise be effective.

On the other hand, when we checked the candidates detected in STEP 2 with the engineers in charge of the real project, we found that the candidates had not been recognized as problems in actual operation. The meeting minutes available in this study could not fully reflect the overall picture of the requirements decisions in the project. This gap is considered to have led to the finding that the detected candidates were not actual problems. This indicates that what the framework detects are merely candidates identified in light of the available related artifacts, and that their interpretation must take the comprehensiveness of the artifacts into account. Therefore, to operate the framework as a completeness review method in real projects, it is important to establish mechanisms for adequately retaining and sharing the necessary information on requirements.

Finally, from the perspective of execution time, as shown in Section V-D, the LLM completed STEP 1 and STEP 2 in around 20 minutes in every trial, with only small differences across the trials. In practice, requirements reviews generally require considerable effort, including preparing review meetings, coordinating participants’ schedules, running the meetings, and organizing the results. At the company that provided the project data, one review cycle is estimated to require approximately ten person-days of effort. Although we did not directly compare the LLM-based process with manual work, the LLM completed the process in approximately 18.5 minutes on average, suggesting that it has the potential to contribute to reducing the review effort.

B. Threats to Validity

This section discusses the threats to validity from the perspectives of internal, construct, and external validity.

1) *Threats to Internal Validity*: The threat concerns the reliability of the ground-truth data used to evaluate the requirements correspondence lists. In the experiment, the authors manually established the correspondences between the SRS and the IRM obtained in each trial and used them as the ground truth for computing recall and precision. Evaluating against the IRM is itself consistent with the purpose of this evaluation—namely, measuring the usefulness of the LLM in establishing correspondences in STEP 2, given the output of STEP 1—and we consider it valid. However, because creating the ground truth involves human interpretation, errors in the correspondences and ambiguity in the judgment criteria may have been introduced into the ground-truth data. In particular, for requirements whose granularity admits multiple interpretations, such as the “semantically decomposable requirements” described in Section V-A, the correspondence judgments may vary depending on the creator. The computed recall and precision may therefore be affected by such errors in the ground truth and may not accurately reflect the true values.

2) *Threats to Construct Validity*: The threat concerns whether the evaluation metrics used in this study adequately reflect the practical usefulness of the proposed framework. As described in Section V-C, the candidates detected in the experiment had not been recognized as problems in the real project. This is because the precision in Section V-B was computed against the IRM, whereas the presence or absence of problems in actual operation is judged against the overall picture of the requirements decisions, which includes information beyond the minutes. High recall and precision therefore do not necessarily imply the practical usefulness of the framework.

3) *Threats to External Validity*: The threats here concern the generalizability of the experimental results. The first threat is that the implementation of the framework in this study depends on the characteristics of the target project. In the experiment, the minutes and the SRS were clearly separated and organized in chronological order, which made the implementation of STEP 1 and STEP 2 relatively straightforward. However, the structure and description style of related artifacts may differ substantially across projects. When applying the framework to other projects, the input format and prompt design for the LLM must therefore be adjusted according to the artifact structure and description style of the target project.

The second threat is that this study examined completeness only for functional requirements. Non-functional requirements and constraints have higher levels of abstraction and greater diversity of expression, and thus differ in nature from functional requirements. Whether the results of this study apply equally to all requirements, including non-functional requirements and constraints, requires further investigation.

The third threat is that the experimental results were obtained with a specific LLM (GPT-5). Because we did not conduct comparative experiments with multiple LLMs, whether

other LLMs can achieve similar usefulness as a means of implementing the framework remains future work.

VII. CONCLUSION

In this paper, we proposed a framework for supporting the completeness review of SRSs on the basis of their consistency with related artifacts, addressing the challenge of ensuring the quality of SRSs in software development. We also presented an LLM-based implementation of the framework and evaluated its applicability through an experiment using data from a real software development project.

Through the experiment, we confirmed that the proposed framework can be executed on real project data. In the correspondence judgment against the IRM, high recall and precision were obtained stably across the trials, confirming the usefulness of an LLM as a means of implementing the framework. In addition, the entire process was completed in around 20 minutes in every trial, confirming that the LLM-based implementation of the framework has the potential to contribute to reducing the time cost of the completeness review. On the other hand, the candidate missing requirements and candidate unsubstantiated requirements detected in the experiment had not been recognized as problems in the real project. These candidates stemmed from decisions made through informal communication that was not recorded in the meeting minutes. This result highlights the importance of adequately recording requirements-related decisions in the related artifacts when applying the framework.

As discussed above, an artifact-based completeness review depends on the scope of the information contained in the accessible artifacts. The proposed framework is therefore positioned not as a replacement for existing review processes in practice, but as a supplementary review method that checks completeness using formalized artifacts.

Future work includes evaluating the applicability of the framework to all types of requirements, including non-functional requirements and constraints, rather than functional requirements alone; evaluating the generalizability and robustness of the results through comparative experiments with different projects and multiple LLMs; and improving the reliability of the evaluation by having multiple evaluators create the ground-truth data.

VIII. ACKNOWLEDGMENT

This work was partly supported by JSPS KAKENHI Grant Number 25K15056.

REFERENCES

- [1] “ISO/IEC/IEEE International Standard - Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” ISO/IEC/IEEE 29148:2018(E), pp. 1–104, 2018.
- [2] C. Denger and T. Olsson, “Quality Assurance in Requirements Engineering,” in *Engineering and Managing Software Requirements*. Springer, 2005, pp. 163–185.
- [3] T. Tamai and M. I. Kamata, “Impact of Requirements Quality on Project Success or Failure,” in *Design Requirements Engineering: A Ten-Year Perspective*. Springer, 2009, pp. 258–275.

- [4] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "AI-based Question Answering Assistance for Analyzing Natural-language Requirements," in *Proc. International Conference on Software Engineering (ICSE)*, 2023, pp. 1277–1289.
- [5] A. A. Porter, L. G. Votta, and V. R. Basili, "Comparing Detection Methods for Software Requirements Inspections: A Replicated Experiment," *IEEE Transactions on Software Engineering*, vol. 21, no. 6, pp. 563–575, 1995.
- [6] J. Lin, A. Poudel, W. Yu, Q. Zeng, M. Jiang, and J. Cleland-Huang, "Enhancing Automated Software Traceability by Transfer Learning from Open-World Data," arXiv preprint arXiv:2207.01084, 2022.
- [7] M. Krishna, B. Gaur, A. Verma, and P. Jalote, "Using LLMs in Software Requirements Specifications: An Empirical Evaluation," in *Proc. International Requirements Engineering Conference (RE)*, 2024, pp. 475–483.
- [8] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.
- [9] "ISO/IEC/IEEE International Standard - Systems and software engineering — Software life cycle processes," ISO/IEC/IEEE 12207:2017(E), pp. 1–158, 2017.
- [10] D. M. Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafrá, A. Vetró, T. Conte, M.-T. Christiansson, D. Greer, C. Lassenius, T. Männistö, M. Nayabi, M. Oivo, B. Penzenstadler, D. Pfahl, R. Prikladnicki, G. Ruhe, A. Schekelmann, S. Sen, R. Spinola, A. Tuzcu, J. L. de la Vara, and R. Wieringa, "Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice," *Empirical Software Engineering*, vol. 22, no. 5, pp. 2298–2338, 2017.
- [11] M. E. Fagan, "Design and code inspections to reduce errors in program development," *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976.
- [12] B. W. Boehm, "Verifying and Validating Software Requirements and Design Specifications," *IEEE Software*, vol. 1, no. 1, pp. 75–88, 1984.
- [13] D. Zowghi and V. Gervasi, "On the interplay between consistency, completeness, and correctness in requirements evolution," *Information and Software Technology*, vol. 45, no. 14, pp. 993–1009, 2003.
- [14] A. Ferrari, F. dell'Orletta, G. O. Spagnolo, and S. Gnesi, "Measuring and Improving the Completeness of Natural Language Requirements," in *Proc. Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2014, pp. 23–38.
- [15] Y. Zhang, H. Jin, D. Meng, J. Wang, and J. Tan, "A comprehensive survey on automatic text summarization with exploration of LLM-based methods," *Neurocomputing*, vol. 663, p. 131928, 2026.
- [16] T. Oami, Y. Okada, and T. Nakada, "Performance of a Large Language Model in Screening Citations," *JAMA Network Open*, vol. 7, no. 7, p. e2420496, 2024.
- [17] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, "Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study," *Automated Software Engineering*, vol. 32, no. 1, p. 26, 2025.
- [18] T. Ahmed and P. Devanbu, "Few-shot training LLMs for project-specific code-summarization," in *Proc. International Conference on Automated Software Engineering (ASE)*, 2022, pp. 177:1–177:5.
- [19] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.
- [20] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, "Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review," arXiv preprint arXiv:2509.11446, 2025.
- [21] L. Pasquale, A. Ragone, E. Piemontese, and A. Amiri Darban, "Exploring the Use of LLMs for Requirements Specification in an IT Consulting Company," in *Proc. International Requirements Engineering Conference (RE)*, 2025, pp. 389–399.
- [22] A. E. Gärtner and D. Göhlich, "Automated requirement contradiction detection through formal logic and LLMs," *Automated Software Engineering*, vol. 31, no. 2, p. 49, 2024.
- [23] A. Fantechi, S. Gnesi, and L. Semini, "Combining Established and Emerging Techniques to Detect Inconsistencies in Requirements," in *Proc. International Requirements Engineering Conference (RE)*, 2025, pp. 519–526.
- [24] S. Bashir, A. Ferrari, M. A. Khan, P. E. Strandberg, Z. Haider, M. Saadatmand, and M. Bohlin, "Requirements Ambiguity Detection and Explanation with LLMs: An Industrial Study," in *Proc. International Conference on Software Maintenance and Evolution (ICSME)*, 2025, pp. 620–631.
- [25] O. Gotel and C. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. International Conference on Requirements Engineering (RE)*, 1994, pp. 94–101.
- [26] B. Regnell, R. B. Svensson, and K. Wnuk, "Can We Beat the Complexity of Very Large-Scale Requirements Engineering?" in *Proc. Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2008, pp. 123–128.
- [27] K. Wnuk, B. Regnell, and C. Schrewelius, "Architecting and Coordinating Thousands of Requirements – An Industrial Case Study," in *Proc. Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2009, pp. 118–123.
- [28] P. Ghazi and M. Glinz, "Challenges of working with artifacts in requirements engineering and software engineering," *Requirements Engineering*, vol. 22, no. 3, pp. 359–385, 2017.